MMI 714 Term Project Report: Adapting Conditional Diffusion Models for Monocular Depth Inpainting of Transparent Objects in Resource-Constrained Environments

Çağdaş Güven
Number: 2738938
Middle East Technical University
Ankara, Türkiye
cagdas.guven@metu.edu.tr

***Abstract*—Transparent and reflective objects decrease the performance of standard perception pipelines as a result of light interaction. Causing light to pass through or reflect of surfaces, results in optical and geometric depth loss. This project is an approach to challenge of recovering monocular depth data by inpainting such transparent objects in hardware-resource constrained environment where high-end compute for latent diffusion is unavailable. I propose a lightweight, Pixel-Space Conditional Diffusion framework that can hallucinate missing geometry by help of RGB data and guidance. Essentially treating depth estimation as a generative inpainting task. My approach adapts a state of the art method from DITR architecture by replacing the heavy Latent Diffusion backbone with hybrid conditional U-Net operating directly in pixel space, guided by robust Canny boundary map extraction and Cross-Attention for hybridization mechanisms. This architectural change preserves high-frequency structural details of characteristic transparend features that are otherwise destroyed in low resolution data latent spaces. Ablation studies on the lower resolution ClearGrasp dataset demonstrates that my spatially exclusive conditioning strategy can outperform baseline models like DeepCompletion achieving 30 percent reduction in RMSE in comparison. Demonstrates it can provide geometric consistency recovering invisible object boundaries.**

I. Introduction

Perception of transparent objects is an important area of bottleneck in sensors. Especially if I implement sensors in robotic systems, manipulation in a 3D environment becomes a challenge. Standard RGB-D cameras with active stereo or Time of Flight technologies rely on projected infrared patterns to estimate depth. The infrared interacting with transparent surfaces passes through the object. Resulting sensor measurement to become background depth rather than object surface. In the Diffusion-Based Depth Inpainting for Transparent and Reflective Objects DITR [1] paper this is called Optical Depth Loss. Also specular reflections and translation between stereo sensors create phantom depth measurements. This is result of occlusion and parallax errors and it is seen as voids around the 3D object edges. DITR classifies this phenomenon as Geometric Depth Loss.

To overcome unexpected collisions in robotic systems, it is necessary to address these errors with a capable system that can hallucinate valid 3D geometry. Improving raw depth data with auxiliary raw image data satisfies the compactness intuition of a mobile system designer.

Previous solutions prior to development of diffusion models and latent diffusion models relied on complex optimization pipelines. ClearGrasp [2] requires estimation of surface normals, occlusion boundaries to solve global optimization problems. But these estimations are preprocessed from raw data therefore system computational complexity is high. Results may satisfy industrial needs of accurate depth information. After 2020, with the publication of Denoising Diffusion Probabilistic Models (DDPM) [3] multimodal distribution estimation required less preprocessed information. It is capable of high frequency geometric details by learning data distribution p(x). SOTA methods like DITR [1], TransDiff [6] and Marigold [5] leverage a method introduced in the 2017 VQ-VAE paper [4]. A latent space generation method introduced in variational autoencoder paper allows diffusion models to be computationally lighter as a result of utilization of smaller latent dimension. A heavy autoencoder VQ-VAE performs compression of high resolution images into smaller latent space and utilizes foundation models like Segment Anything [8] for semantic guidance. Nevertheless this pipeline requires high end computational resources (e.g. A100 GPUs) that are often inaccessible for edge robotics applications. Instead of Latent compression and concatenation of dependent signals in the latent space, Pixel-space conditional diffusion architecture is investigated in this project. By bypassing the VQ-VAE bottleneck and implementing dual diffusional model branch from DITR exclusive contextual guidance combining RGB context with edge aware constraints (M_{DA} and M_{RGB}) I demonstrate that effective depth inpainting is possible without the computational overhead of latent space modeling.

II. Related Work

1) Transparent Object Depth Estimation: : As mentioned in first chapter early approaches focused on depth completion using localized heuristics. Cleargrasp [2] utilizes convolutional network to predict surface normals, occlusion boundaries, transparency masks and performs global optimization step to reconstruct depth data. TranspareNet is a regression-based CNN which attempts to predict depth directly but suffers from averaging effect of MSE loss, resulting loss of geometric boundaries.

2) Diffusion Models in Vision: Denoising Diffusion Probabilistic Models (DDPM):: DDPM [3] introduced generating data by reversing a gradual noise-addition process. Capable of high-fidelity generation but pixel space DDPMs are computationally intensive. VQ-VAE [4] and subsequent Latent Diffusion Models (LDM) addressed this by compressing images into a discrete latent space. Minimized data loss and allowed computationally lighter manipulation of latent space.

3) Diffusion for Depth Estimation: : Marigold [5] finetunes pretrained LDMs (like Stable Diffusion) for monocular depth estimation. Provides SOTA quantitative results but requires heavy pretrained backbones. D3RoMa [7] treats depth estimation problem as stereo disparity problem. Using specific stereo camera setups uses diffusion to predict disparity maps. Transdiff [6] uses Transformer based diffusion architecture and uses semantic segmentation and surface normals as conditions.

4) DITR and project adaptation: : My work is mostly related to DITR [1] it proposes two stage framework: first a region proposal network extracts mask for only transparent areas excluding other optical phenomenon such as mirrors.DITR utilizes special neural network called TrosNET but I utilize provided mask from ClearGrasp dataset. Then region mask divides data into dual branch Latent Diffusion model and separately handles optical and geometric loss. DITR relies on high resolution (1080p) inputs compressed via encoder network to the 240 by 135 pixels. Also in the depth and RGB data another neural network extracts boundary maps. Researchers utilized Segment Anything Model [8] for this job. For the pixel space of 128x128 I tested SAM, PiDiNet [9] and Canny. Also Global Contextual Guidance strategy to anchor the diffusion process is proposed and investigated which was an approach to lack of prior knowledge of the masked regions during training process.

III. Methodology

A. Problem Formulation

Given: RGB image $I_{RGB} \in \mathbb{R}^{H\times W\times 3}$: Contains visible appearance of transparent objects Raw depth $D_{raw} \in \mathbb{R}^{H\times W}$: Corrupted depth map with missing/incorrect values in transparent regions Segmentation mask $M \in \{0,1\}^{H\times W}$: Binary mask where $M = 1$ indicates transparent object regions

Our objective is to predict the complete depth map $D_{complete}$ that accurately represents the true geometry of transparent objects.

B. Dataset Preparation

1) ClearGrasp Dataset: I utilize the ClearGrasp dataset. The dataset provides paired ground truth for transparent objects. Training set provides synthetically created setups of images containing one or more transparent object, mask, RGB image. The test set includes real world data correspondence. Ground truth is created by replacing transparent objects with painted version.

Training set: 45,000 synthetic images across multiple object classes Test set: 286 real-world RGB-D images captured with RealSense D415/D435 cameras

2) Data Preprocessing: All images are resized to 128×128 pixels to accommodate GPU memory constraints (RTX 4060 8GB VRAM). Depth values are normalized from meters to the range $[-1, 1]$:

$$D_{norm} = 2 \cdot \frac{D - D_{min}}{D_{max} - D_{min}} - 1$$

where $D_{min} = 0.0$m and $D_{max} = 5.0$m define the valid depth range.

3) Input Corruption Simulation: During training, I simulate sensor failure by zeroing out synthetic depth values in masked (transparent) regions:

$$D_{input} = D_{norm} \cdot (1 - M)$$

This creates the corrupted input that mimics real-world sensor behavior where depth is missing inside glass objects.

C. Network Architecture

1) Two-Branch Pipeline : Following the DITR framework, I employ a two-branch architecture:

Optical Branch : Handles depth inpainting inside the transparent object mask. This branch learns to hallucinate depth for regions where the sensor completely fails.

Geometric Branch: Refines depth in the background regions outside the mask. This branch corrects minor depth errors and ensures smooth transitions.

2) Conditional U-Net Architecture : Each branch uses an identical Conditional U-Net architecture with the following specifications:

```
class ConditionalUNetDepth(nn.Module):
     base_channels: int = 64,
     channel_mults=(1, 2, 4, 8),
     num_res_blocks: int = 2,
     time_emb_dim: int = 256,
     cond_channels: int = 5,
     #RGB(3)+raw_depth(1)+guidance(1)
```

3) Key architectural components: Sinusoidal time embedding: Encodes diffusion timestep t into a 256-dimensional vector ResNet blocks: Each block contains GroupNorm, SiLU activation, and 3×3 convolutions with residual connections Cross-attention layers: Enable the model to attend to conditioning features at multiple resolutions Encoder-decoder structure: 4 resolution levels with channel multipliers (64, 128, 256, 512)

Conditioning mechanism: The 5-channel conditioning tensor is constructed as: $C = [I_{RGB}, D_{input}, G]$

where G represents the guidance map ($M_{DA} or M_{RGB}$ depending on the branch).

4) Conditioning Feature Encoder: A separate encoder processes the conditioning tensor through the same resolution levels as the main U-Net, producing multi-scale feature maps for cross-attention:

Condition encoder: encode $cond_{rgbd}$ to feature maps at each resolution

```
self.cond_in = nn.Conv2d(
   self.in_channels_cond,
   init_dim, 3, padding=1
)
self.cond_downs = nn.ModuleList([])

for i, mult in enumerate(channel_mults):
   out_ch = base_channels * mult
   # Break long arguments to next line
   self.cond_downs.append(ResnetBlock(
      cond_ch, out_ch, time_emb_dim=None
   ))

   if i != len(channel_mults) - 1:
      self.cond_downs.append(
         Downsample(cond_ch, cond_ch)
      )
```

5) Guidance Map Generation: M_{DA} (Depth-Aware Guidance) for Optical Branch:

The M_{DA} guidance map captures edges visible in RGB but not in depth, highlighting glass boundaries

$M_{DA} = M_{RGB} - M_D$

where: M_{RGB}: Canny edges from the RGB image M_D: Canny edges from the zeroed depth map

This difference operation identifies contours of transparent objects that the depth sensor cannot perceive.

M_{RGB} Guidance for Geometric Branch:

The M_{RGB} guidance consists purely of RGB edges multiplied with 1-mask:

$M_{RGB} = \text{Canny}(I_{RGB}) \times (1 - mask)$

This provides structural guidance for refining background depth.

Edge Detection Configuration

```
class GuidanceConfig:
  edge_detector: Literal['canny',
      'sam', 'pidinet'] = 'canny'
  rgb_canny_thresh1: int = 100
  rgb_canny_thresh2: int = 200
  depth_canny_thresh1: int = 30
  depth_canny_thresh2: int = 100
  depth_max_distance: float = 10.0
  depth_use_inverse: bool = True
  # Use 1/z for better near-object edges
```

For robust depth edge detection, I optionally use inverse depth (disparity) representation:

$$D_{inv} = \frac{1}{D + \epsilon}$$

This emphasizes near-object boundaries and compresses far-field variations.

6) Diffusion Process: The forward diffusion(noising) process gradually adds Gaussian noise to the clean depth map over $T = 1000$ timesteps:

$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I})$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ and $\alpha_t = 1 - \beta_t$.

I use a linear noise schedule: $\beta_t = \beta_{start} + \frac{t}{T}(\beta_{end} - \beta_{start})$ with $\beta_{start} = 10^{-4} and \beta_{end} = 2 \times 10^{-2}$.

The reverse process(denoising) iteratively denoises from random noise $x_T \sim \mathcal{N}(0, \mathbf{I})$:

$p_\theta(x_{t-1}|x_t, C) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t, C), \sigma_t^2\mathbf{I})$

The model predicts the noise ϵ_θ rather than directly predicting x_0:

$\mu_\theta = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t, C)\right)$

7) Training Objective: Train occurs using the simplified denoising score matching objective:

$\mathcal{L} = \mathbb{E}_{x_0,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(x_t, t, C)\|^2\right]$

Both branches are trained on the mask and 1-mask conditions. The conditioning (different for each branch) is what differentiates their learned behaviors. Also no masked global condition is investigated and discussed in the discussion section.

```
def p_losses(self,
   batch, *, noise=None) -> torch.Tensor:
x_start = batch["pixel_values"]
cond = batch["conditioning"]

if noise is None:
  noise = torch.randn_like(x_start)

t = torch.randint(0,
   self.timesteps, (b,), device=device)
x_noisy = self.q_sample(
   x_start=x_start, t=t, noise=noise)

model_out = self.model(x_noisy, cond, t)
loss = ((noise - model_out) ** 2).mean()

return loss
```

8) Training Procedure: Data Pipeline

```
class DepthDataConfig:
root: str
size: Tuple[int, int] = (128, 128)
depth_minmax: Tuple[float, float] = (0.0, 5.0)
branch: Literal["optical", "geometric"] = "optical"
include_guidance: bool = True
use_precomputed_guidance: bool = True
```

Guidance maps are pre-computed using script to accelerate training.

9) Training Configuration:

Parameter	Value
Batch size	256
Learning rate	1×10^{-4}
Optimizer	AdamW (weight decay = 0.01)
Epochs	30
Timesteps	1000
Image size	128 × 128
GPU	NVIDIA RTX 4060 (8GB VRAM)

Training Loop:

```
# Training with mixed precision for memory efficiency
scaler = torch.amp.GradScaler('cuda')

for epoch in range(EPOCHS):
  for batch in dataloader:
     with torch.amp.autocast('cuda'):
        loss = diffusion.p_losses(batch)

     optimizer.zero_grad()
     scaler.scale(loss).backward()
     scaler.unscale_(optimizer)
     torch.nn.utils.clip_grad_norm_(
           model.parameters(), 1.0)
     scaler.step(optimizer)
     scaler.update()
```

10) Inference Pipeline: Branch-Specific Conditioning

During inference, each branch receives appropriately conditioned inputs:

Optical Branch: $C_{opt} = [I_{RGB}, D_{raw} \cdot (1 - M), M_{DA}]$

Geometric Branch: $C_{geo} = [I_{RGB}, D_{raw} \cdot (1 - M), M_{RGB}]$

11) Sampling and Merging: Both branches perform 1000-step DDPM sampling:

```
def run_ditr_inference(sample,
guidance, opt_diff, geo_diff, cfg, device):
# Prepare conditioning
cond_opt = prepare_conditioning(sample,
guidance, "optical", cfg)
cond_geo = prepare_conditioning(sample,
guidance, "geometric", cfg)

# Run both branches
pred_opt = opt_diff.sample(cond=cond_opt,
shape=shape, device=device)
pred_geo = geo_diff.sample(cond=cond_geo,
shape=shape, device=device)

# Merge: optical inside mask, geometric outside
merged = pred_opt * mask + pred_geo * (1.0 - mask)

return merged
```

The final depth map is constructed by:

$$D_{final} = D_{opt} \cdot M + D_{geo} \cdot (1 - M)$$

12) Evaluation Metrics: Following the DITR paper, evaluate conducted using standard depth estimation metrics computed only on pixels with valid ground truth:

Error Metrics

Root Mean Square Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(D^i_{pred} - D^i_{gt})^2}$$

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|D^i_{pred} - D^i_{gt}|$$

Mean Relative Error (REL):

$$\text{REL} = \frac{1}{n}\sum_{i=1}^{n}\frac{|D^i_{pred} - D^i_{gt}|}{D^i_{gt}}$$

Threshold Accuracy (δ Metrics)

$$\delta_\tau = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\left[\max\left(\frac{D^i_{pred}}{D^i_{gt}}, \frac{D^i_{gt}}{D^i_{pred}}\right) < \tau\right]$$

Reported thresholds are $\delta_{1.05}$, $\delta_{1.10}$, and $\delta_{1.25}$.

Metrics are computed for both masked regions and complete image, for this report masked region is focused.

IV. Experiments and Results

Ablation studies has been conducted to evaluate impact of edge guidance mapping, loss function design, architectural choices on depth inpainting performance. Experiments mainly conducted on single NVIDIA RTX 4060 (8GB VRAM)(mobile) along with Google Colab provided A100 GPU at a resolution of 128x128. Evaluation conducted on a RealWorld Test set. Training set test is also conducted using original synthetic data as ground truth

A. Ablation Study 1: Old Architecture with PiDiNet Mapping

Configuration: Architecture: Pixel-Space Conditional U-Net (Input Concatenation Only). Guidance: PiDiNet Edge Detection. Mapping Logic: $M_{DA} = M_{RGB} \cap M_{Depth}$ (Intersection). Conditioning: Mutually exclusive masking (Optical model sees only glass edges; Geometric model sees only background). Results: The PiDiNet-based guidance produced extremely sparse feature Fig. 1maps at 128×128 resolution. Due to alignment errors inherent in low-resolution downsampling, the intersection between RGB and Depth edges often resulted in empty maps ($M_{DA} \approx 0$). Consequently, the model received almost no structural guidance. The resulting depth maps were blurry and failed to capture the geometry of the transparent objects, performing only marginally better than a baseline with no guidance.

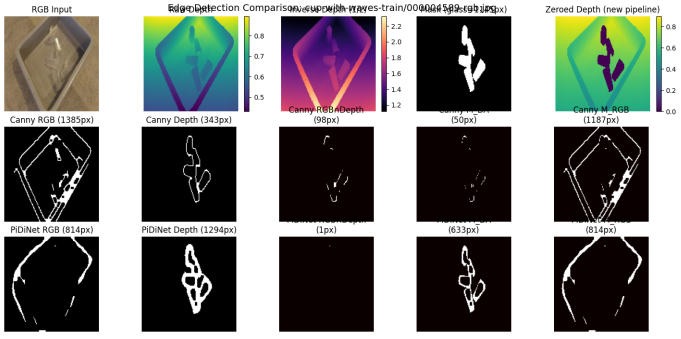

Fig. 1. Pidinet Boundary detection. Considering zeroed out masked zones, extraction is biased towards mask edges unlike canny version implemented in Fig 2

B. Ablation Study 2: Old Architecture with Canny Mapping (Qualitative Only)

Configuration: Architecture: Same as Study 1. Guidance: Canny Edge Detection (Robust at low resolution). Mapping Logic: $M_{DA} = M_{RGB} \setminus M_{Depth}$ (Difference). Results: Quantitative metrics for this configuration are unavailable. However, qualitative analysis indicated that switching to Canny provided

denser boundary maps Fig. 2 than PiDiNet. Despite this, the model produced "Floating Ghost" artifacts, generating glass shapes that were geometrically plausible but spatially misaligned (e.g., floating at incorrect depths). This motivated the architectural shift to Cross-Attention in subsequent studies.

C. Ablation Study 3: New Architecture with Mask-Focused Conditioning (Best Model)

Configuration: Architecture: New Hybrid U-Net (Input Concatenation + Cross-Attention). Guidance: Canny Edge Detection. Conditioning: Spatially Exclusive (Optical model receives M_{DA} only inside the mask; Geometric model receives M_{RGB} only outside the mask). Results: This configuration achieved the best performance. By focusing the optical branch strictly on the masked region and the geometric branch strictly on the background, the models specialized effectively. As shown in Table I (Row 5), Fig 5 this configuration achieved the lowest RMSE of 0.0970m on the masked region and the highest accuracy metrics. Sanity Check on Training Data Fig 3: To verify the model's capacity, I tested this best configuration on samples from the synthetic training set. The model successfully inpainted the zeroed-out regions, producing depth values that closely matched the ground truth (Input Mask Area $\Delta < 1.05 = 0$ vs Prediction $\Delta < 1.05 \approx 99\%$), confirming that the model learned the underlying distribution of the synthetic data.

D. Ablation Study 4: New Architecture with Global Contextual Guidance

Configuration: Architecture: New Hybrid U-Net. Guidance: Canny Edge Detection. Conditioning (Optical): Combined Map: $Condition = (Mask \times M_{DA}) + ((1 - Mask) \times M_{RGB})$. Conditioning (Geometric): Full M_{RGB} (Global context). Results: Contrary to my hypothesis, providing global context to the optical branch degraded performance compared to Study 3. The model struggled to prioritize the fine glass edges (M_{DA}) when simultaneously presented with the strong background edges (M_{RGB}). This suggests that for low-resolution training from scratch, strict separation of tasks (Mask-Focused) is more effective than global context fusion.

E. Ablation Study 5: No Mapping (Control)

Configuration: Architecture: New Hybrid U-Net. Guidance: None (Conditioning channel set to 0). Results: Fig 4The model failed to generate coherent object structures, outputting amorphous blobs in the transparent regions. This confirms that edge guidance (M_{DA}) is not just helpful but critical for defining the geometry of invisible objects. The model generates noisier version of reconstruction and for masked area qualitative result shows mean noise

F. Quantitative Comparison

Method

Table I: Quantitative comparison on the ClearGrasp Real-World Test Set. Note: The "Input" RMSE appears artificially low because it ignores missing data pixels (holes), whereas our method fills these holes, accumulating error over a larger set of valid pixels.

Our best method (Study 3) Fig 5outperforms DeepCompletion significantly in accuracy ($\delta < 1.05$) and error metrics (RMSE/REL), demonstrating the efficacy of the Spatially Exclusive Masking strategy for this architecture.

V. Discussion

A. Architectural Evolution:

From Input Concatenation to Cross-Attention Initial experiments with standard Pixel-Space U-net using simple input concatenation showed a critical limitation, especially after transitioning from PiDiNet to Canny edge detection. Canny provided a denser guidance signal because it can capture internal reflections and texture boundaries that could not be detected by PiDiNet in the low resolution dataset. Initial improvement after canny showed that high frequency guidance map may become diluted as the network downsampled features leading to floating ghost-like objects in the masked area. A geometry is generated but spatially makes no sense. This would result in a scenario of direct implementation of DITR leaving the heavy work to mask extractor network. To address this, combining input concatenation with cross-attention mechanism at every resolution level, a hybrid U-Net Architecture is implemented. Thus dense edge guidance $(M_{da} and M_{rgb})$ and RGB context are reinjected deep into the bottleneck. Resulting in the significant performance jump observed in the study 3.

B. The Efficacy of Spatially Exclusive Conditioning

During testing I hypothesised about providing "Global Contextual Guidance". In this hypothesis, during training, model would be conditioned not only the exclusive boundary map multiplied by area of interests. (mask for optical model, 1-mask for geometric model) therefore model would be provided a prior knowledge corresponding to rgb value for the depth unlike Spatially Exclusive Conditioning where zeroed out masked regions are not provided with boundary map condition for the background. Assumption was introduction of a dependence with RGB information and depth via boundary map. But results showed this actually decreased performance of the model. For the best model optical branch sees only mask region boundary maps along with rgb channels, similar with geometric channel seeing only 1-mask region boundary map along with rgb channels.

C. Loss Function Dynamics and Normalization Artifacts

I extensively investigated loss function designs, including Weighted Masking (60 Glass / 40 Background) and Hard Masking. Hard Masking: Calculating loss only on the masked region caused instability and gradient explosions, likely because the model lost its global reference frame (the "anchor" effect). Global MSE: The most stable configuration proved to be a standard Global MSE loss, relying on the conditioning (not the loss function) to differentiate the branches. I also observed that input normalization to $[-1, 1]$ introduced accuracy loss areas close to maximum and minimum depth

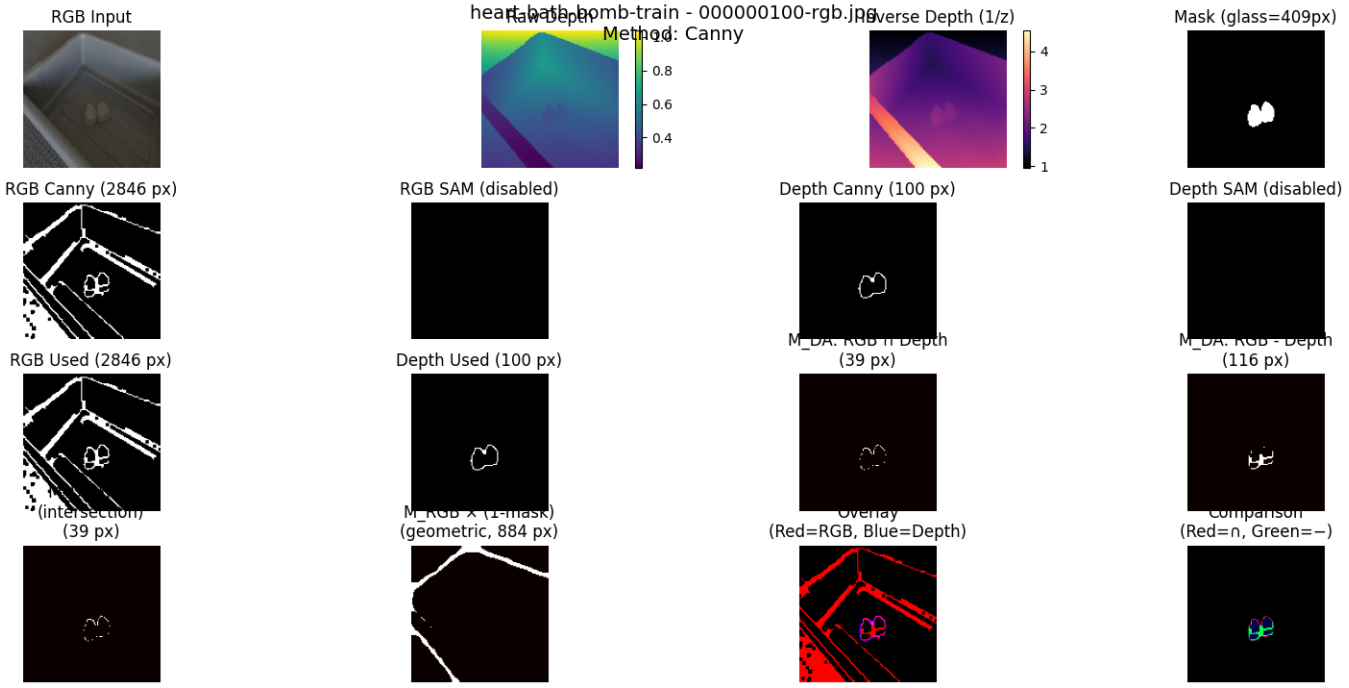

Fig. 2. Implementation of Canny boundary detection map. Provides more robust boundaries (eg green colored edges showing edges of transparent object)

Method	RMSE (m)	MAE	REL	$\delta < 1.05$	$\delta < 1.10$	$\delta < 1.25$
Input (Raw w/ No Boundary Map)	0.1250	0.1196	0.1995	18.97	31.60	58.03
DeepCompletion[1]	0.209	0.207	0.396	34.61	52.79	71.32
Study 1 (Old+PiDiNet)	0.1175	0.0985	0.1641	27.87	44.06	68.72
Study 4 (New+Global)	0.1152	0.0968	0.1635	26.04	44.27	69.10
Study 3 (New+Exclusive)	0.0970	0.0816	0.1401	20.51	41.37	82.78
DITR	0.019	0.012	0.030	85.11	94.20	98.92

TABLE I

TABLE REPRESENTS STUDY RESULTS ON CLEARGRASP REAL-WORLD TEST SET (MASKED REGION ONLY) AND COMPARISON FROM DITR [1]

```
================================================================
  Training Data Evaluation Results (200 samples)
================================================================

Metric                Input Full    Pred Full   Input Mask    Pred Mask
----------------------------------------------------------------
RMSE                      0.3809       0.0444       1.7764       0.0765
MAE                       0.0923       0.0421       1.7752       0.0724
REL                       0.2185       0.0665       3.4972       0.1317
δ1.05                     0.9526       0.4847       0.0021       0.1725
δ1.10                     0.9526       0.7680       0.0040       0.4631
δ1.25                     0.9527       0.9618       0.0089       0.9011
----------------------------------------------------------------
n_samples                  16384        16383          776          776
================================================================
```

Fig. 3. Figure shows test on training dataset samples using non zeroed masked zones as ground truth. Tested for sanity check.

value . Qualitative results on the training set showed that while the model successfully inpainted the zeroed-out regions, the predicted depth distribution sometimes shifted relative to the input, likely due to the normalization logic disrupting the absolute metric scale. However, the high accuracy at the $\delta < 1.25$ threshold (82) indicates the relative geometry is preserved even if the absolute scale drifts slightly.

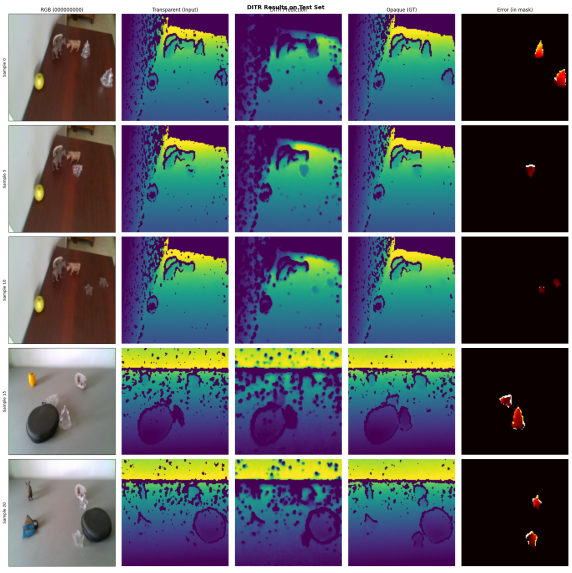

Fig. 4. Shows qualitative results of a model trained on guidance disabled

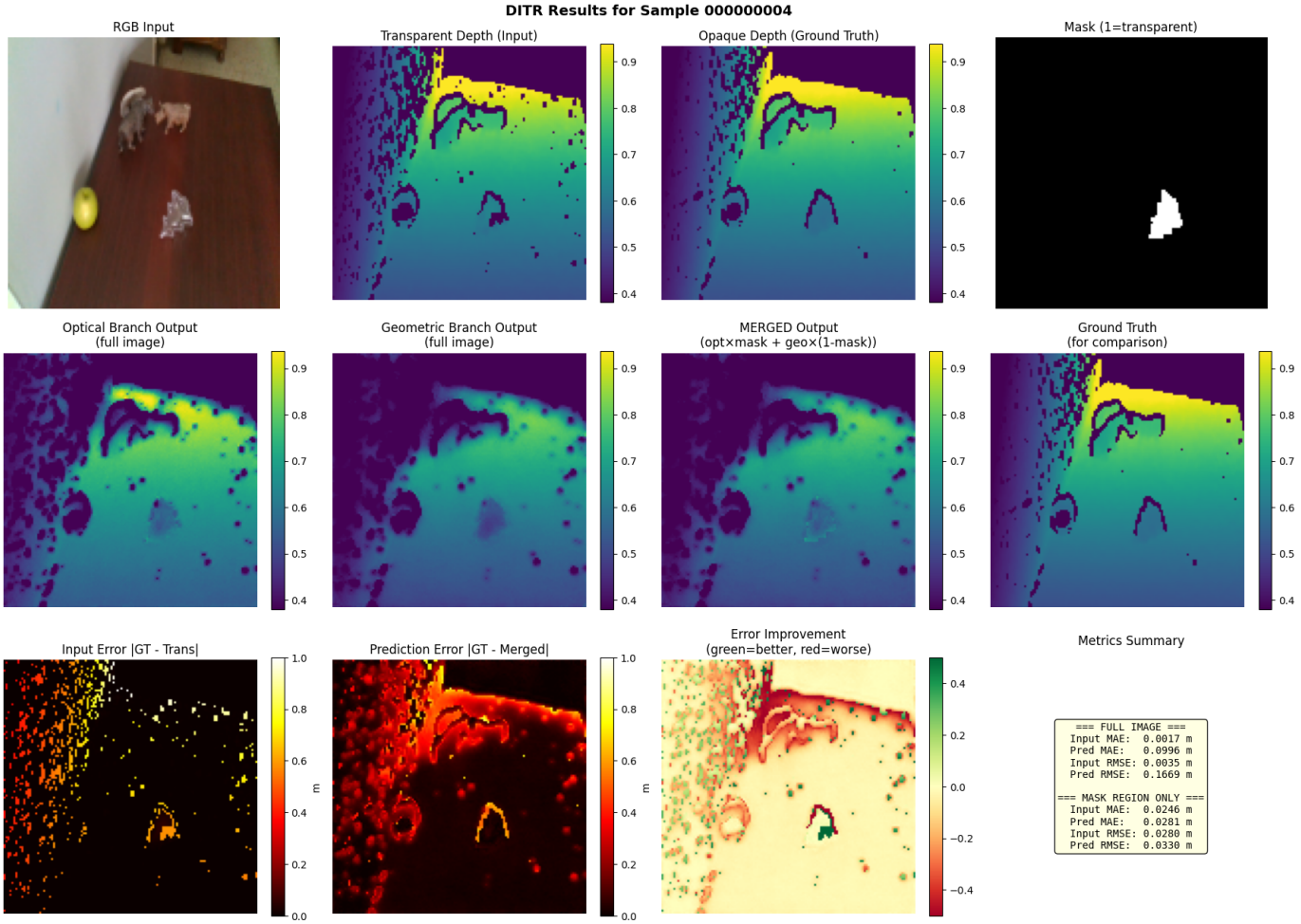

Fig. 5. Qualitative results of best model. Geometric and Optical branches are merged together. Compared to the ground truth green color shows the improvement in masked region

D. The "Safety Net" and Edge Detection Robustness

A key finding was the performance inversion of edge detectors at low resolution (128×128). SAM and PiDiNet: State-of-the-art models like Segment Anything (SAM) and PiDiNet produced noisy or overly sparse maps. At 128p, the semantic features they rely on are often lost, leading to "broken" masks (figure 1) that confuse the diffusion model. Canny Edge Detector: The simpler, gradient-based Canny detector proved far more robust. It consistently captured the high-frequency rims of glass objects that deeper models missed due to downsampling. This validates decision to use Canny mapping as a boundary map extractor. In resource-constrained environments where high-res semantic segmentation is impossible, classical computer vision techniques often outperform heavy deep learning baselines. As for safety net, for the intersection of boundary maps from depth and rgb method explained in the presentation. Pipeline utilized PiDiNet extractor which performed poorly on RGB images resulting lack of boundaries for the masked region. In this situation safety net falls back to depth edges, contradicting idea of DITR. With the utilization Canny, boundary extracted from RGB data treated as reliable source compared to depth data again. E. Latent vs. Pixel Space: A Theoretical Divergence The original DITR [2] and recent works like Marigold [5] rely on Latent Diffusion Models (LDM) compressed via VQ-VAE. While efficient for $1080p$ images, our experiments confirm that at $128p$, Pixel Space can provide depth inpainting provided robust boundary maps and an architecture that carries condition information throughout the network. Furthermore, I identify a theoretical risk in the DITR pipeline: the VQ-VAE decoder is trained on perfect synthetic data. When fed "repaired" latent codes from real-world noisy inputs, the decoder might hallucinate details or smooth out valid geometry based on its learned priors. Our Pixel-Space approach avoids this "Black Box Decoder" issue, ensuring every generated pixel is a direct result of the diffusion process.

VI. Conclusion and Future Work

This work demonstrates that monocular depth inpainting for transparent objects is achievable on consumer-grade hardware by adapting state-of-the-art diffusion architectures to Pixel

Space. I propose a Hybrid Conditional U-Net that leverages Cross-Attention and robust Canny-based edge guidance to recover missing geometry. The Spatially Exclusive Conditioning strategy outperforms global context fusion, achieving an RMSE of 0.0970m on the ClearGrasp Real-World Test Set, significantly surpassing the DeepCompletion baseline. I further highlight the limitations of "blind" downsampling for deep learning edge detectors and the necessity of preserving high-frequency details in the conditioning signal. Future Work: Latent Diffusion with VQ-VAE: With access to high-end compute (e.g., A100/H100), I would implement the full Latent Diffusion pipeline on 1080p images. Specifically, utilizing a pretrained VQ-VAE [4] to handle the resolution upscaling, allowing the diffusion model to focus on structural inpainting in a high-dimensional latent space. Real-World Dataset Training: Currently, the model is trained on synthetic data (ClearGrasp). Future iterations should incorporate real-world datasets like TODD and STD to better capture the noise characteristics of physical sensors, potentially using domain adaptation techniques to bridge the Sim-to-Real gap. Additional Modalities: As mentioned in the C. section in the discussion, the normalization operation needs to be addressed by implementation of robust scaling methods like logarithmic scaling instead of normalization and inverse depth operations.

REFERENCES

[1] T. Sun, D. Hu, Y. Dai, and G. Wang, "Diffusion-Based Depth Inpainting for Transparent and Reflective Objects," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 35, no. 1, pp. 394–404, Jan. 2025.

[2] S. S. Sajjan, M. Moore, M. Pan, G. Nagaraja, J. Lee, A. Zeng, and S. Song, "ClearGrasp: 3D Shape Estimation of Transparent Objects for Manipulation," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 3634–3642.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.

[4] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[5] B. Ke, A. Obukhov, S. Huang, N. Metzger, R. C. Daudt, and K. Schindler, "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 9492–9502.

[6] H. Wang, K. Zhou, B. Gu, Z. Feng, W. Wang, P. Sun, Y. Xiao, J. Zhang, and H. Dong, "TransDiff: Diffusion-Based Method for Manipulating Transparent Objects Using a Single RGB-D Image," arXiv preprint arXiv:2503.12779, 2025.

[7] S. Wei, H. Geng, J. Chen, C. Deng, W. Cui, C. Zhao, X. Fang, L. Guibas, and H. Wang, "D^3RoMa: Disparity Diffusion-based Depth Sensing for Material-Agnostic Robotic Manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[8] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick, "Segment Anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4015–4026.

[9] Z. Su, W. Liu, Z. Yu, D. Hu, Q. Liao, Q. Tian, M. Pietikäinen, and L. Liu, "Pixel Difference Networks for Efficient Edge Detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 5117–5127.